

实验 3：闭环控制

概述

在本实验中，我们将使用**闭环控制**策略来进行轨迹跟踪。在闭环控制系统中，我们可以通过机器人的实时位置反馈，来及时调整当前的机器人的控制输入量。示例脚本中包含一个话题订阅者，用于接收来自 Gazebo 模拟器的实时里程计数据。通过里程计的反馈，机器人可以相应地调整其移动方向，并检查是否已到达目标导航点。

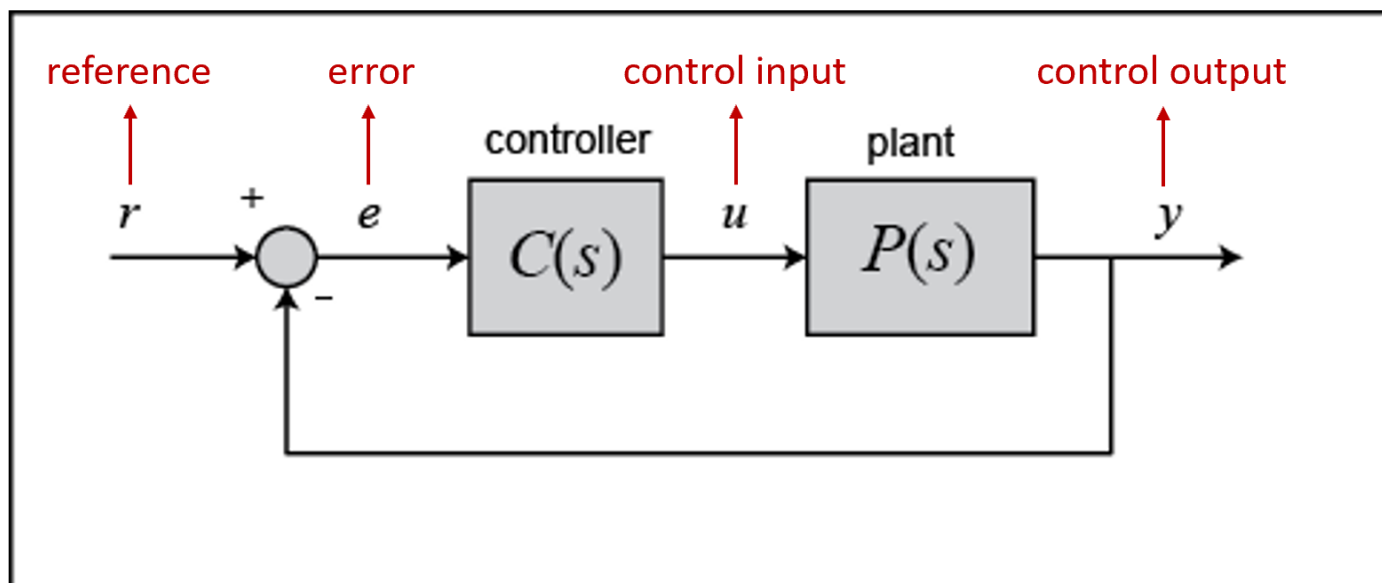
具体来说，这次的任务是实现一个PD控制器（PID控制器的一种变体）来跟踪方形轨迹。与实验 2 一样，导航点仍然是 $[4, 0]$ 、 $[4, 4]$ 、 $[0, 4]$ 、 $[0, 0]$ ，但这次我们要求更高的精度（到达每个导航点的误差小于0.1米）。任务完成后，机器人应停在 $[0, 0]$ 点。请绘制机器人运动轨迹（可以使用另一个提供的 Python 脚本），并在实验报告中讨论你的结果。

作业提交

- 需提交的文件：
 - `closed_loop.py`
 - 提交一份PDF格式的实验报告，报告中包含方形轨迹的绘图和一些必要的讨论
- 评分标准：
 - 机器人可以访问方形轨迹的所有四个导航点（误差 <0.1 米）。将根据访问的导航点数量给予部分分数。

PID控制简介

请看下面的反馈控制图。



- 被控对象（plant）是我们想要控制的系统。在我们的例子中，控制输入 u 是我们发送给机器人的速度指令，控制输出 y 是机器人的当前状态（即2D位姿：x、y、theta），参考值 r 是我们希望机器人达到的期望状态。
- 控制器（controller）是我们需要设计的部分。它的输入是跟踪误差 e ，即期望参考值 r 与实际输出 y 之间的差值，然后根据以下公式计算控制量 u 。（被控对象的控制输入量即是控制器的输出值。）

$$u(t) = k_p e(t) + k_i \int e(t) dt + k_d \frac{de(t)}{dt}$$

- 这个方程揭示了控制器的名称：比例 - 积分 - 微分（PID）控制器，因为它有三项：比例项、积分项和微分项。在每一项中，我们都有一个系数 k 用来和误差（的积分/微分）相乘。 K_p 、 K_i 和 K_d 就是我们需要调整的系数/参数。
- 在数学中，有严格的分析来证明系统的稳定性和收敛性，从而直接计算出最优的 K_p 、 K_i 和 K_d 。（更多信息可以参考[PID控制器设计教程](#)。）然而，在本实验中，手动调整出来的非最优参数足以完成任务。你可以从 K_p 接近 1 的某个值和一个更小的 K_d 的值开始测试。
- 至于误差的积分和导数，在离散系统中，我们可以用求和代替积分，用减法代替导数。

$$u(t) = k_p e(t) + k_i \sum e(t) + k_d \frac{e(t) - e(t - \Delta t)}{\Delta t}$$

- 由于积分项会带来很多负面影响，我们将在PID控制器中去掉这一项，只使用PD控制器。此外，时间间隔 Δt 可以合并到参数 K_d 中。（如果我们以10Hz的频率运行，时间间隔将为0.1秒。）因此，我们可以在本实验中实现以下方程。

$$u(t) = k_p e(t) + k_d (e(t) - e(t - \Delta t))$$

- 我们提供了一个在 Python 中实现的离散 PD 控制器，仅供参考。

编程提示

- 请注意，机器人的方向（theta）在二维平面上的范围是从 `-pi` 到 `pi`。当机器人沿逆时针方向转动并经过负x轴方向时，theta的值将从 `pi` 跳到 `-pi`。这需要妥善处理，否则机器人将原地打转，无法前进。
- 一般来说，PID控制器是用来跟踪某个目标值的（也称为**设定值**），并确保系统能够收敛到这个目标值。例如，控制锅炉系统中的温度。（请注意，这个设定值是一个标量。）
- 在我们的例子中，我们三个变量（x、y、theta）来描述机器人的二维姿态。在同一时间，一个PID控制器中只能有一个变量被设置为跟踪的目标值。如果你想同时跟踪 x、y 和 theta，你将需要三个 PID 控制器。为了完成本实验中的任务，在这三个控制器中，只有跟踪 theta 的控制器是必须的，另外两个跟踪 x 和 y 的控制器是可选的。见下面的示例算法。
- 下面是如何将反馈控制算法应用于航点导航问题的示例。你可以参考这个算法开始你的实现。
 - a. 假设机器人当前的朝向为 θ ，期望朝向为 θ^* ，在X-Y平面上的当前位置为(x, y)，在X-Y平面上的期望位置为(x^* , y^*)。
 - b. 根据 (x, y) 和 (x^* , y^*) 的差值计算移动方向/角度，将其设为期望方向 θ^* 。
 - c. 用设定值 θ^* 和一组参数（`Kp`、`Ki` 和 `Kd`）初始化一个PID控制器。通过PID控制器计算的角速度命令调整机器人的角度/方向。
 - d. 当 θ 达到 θ^* 后，再继续以恒定速度向前移动（或使用另一个PID控制器来控制线速度），并不断调整角度/方向，检查到目标位置 (x^* , y^*) 的剩余距离。
 - e. 当 (x, y) 达到 (x^* , y^*) 后，停止当前运动，并对下一个导航点重复该过程。

示例代码

我们提供了一个名为 `closed_loop.py` 的示例代码。请完成 `run` 函数，`move_to_point` 函数，和控制器中的更新步骤的函数，从而使机器人沿着方形轨迹移动。和之前一样，你可以将其放在 `scripts` 文件夹中，该文件夹位于 `robotics101` 包下。

- 如果缺少 tf 软件包，可以安装 `sudo apt install ros-humble-tf-transformations`
- 在不重启Gazebo的情况下，如果你还想要重新测试脚本，可以使用 `Ctrl+R` 快捷键将机器人重置到原点，然后再次运行脚本。
- 此外，我们还提供了另一个名为 `visualization.py` 的示例代码，可以帮助可视化 csv 文件中的机器人轨迹。